



SOUTENANCE — TITRE PROFESSIONNEL ADMINISTRATEUR SYSTÈME DEVOPS

OpsForge

Console locale de gestion d'incidents et chaîne DevOps de bout en bout

Dyllan Thouvignon

Titre professionnel niveau 6 · TP-01414 · RNCP 36061
Session du 7 septembre 2026 · Liora (ex DataScientest)

Un projet fil rouge indépendant, assumé comme tel

MON PARCOURS

BTS SIO option SLAM,
puis formation DevOps
en alternance.

Support et exploitation,
puis systèmes, supervision
et développement.

POURQUOI CE PROJET

Mes missions touchent à
plusieurs de ces domaines,
mais aucune ne réunit les
trois compétences du titre —
aucune ne porte sur les
containers.

D'où OpsForge, juin-août 2026.

CE QUE ÇA IMPLIQUE

Cahier des charges conçu
par moi, à partir du référentiel.

Aucune donnée ni
infrastructure professionnelle.

Conséquence directe : je tiens tous les rôles — cadrage, réalisation, revue, validation. C'est une limite, et je l'assume explicitement dans le dossier.

Le fil de la présentation

1

Le besoin

Le problème métier
et le cahier des
charges

2

La solution

L'application
et ses règles

3

L'architecture

Ce qui tourne,
et où

4

Les preuves

Trois compétences,
trois démonstrations

5

La recherche

Un blocage réel
et son diagnostic

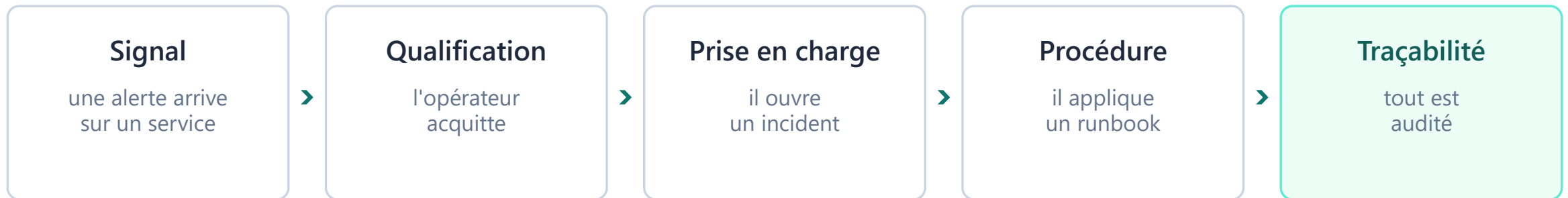
6

Le bilan

Limites assumées
et conclusion

Une règle tout au long : chaque brique présentée a été exécutée et vérifiée — pas seulement écrite.

Un incident, c'est une chaîne de décisions à tracer



Sans outil, trois choses se perdent :

- l'ordre des décisions — qui a fait quoi, dans quel ordre, et pourquoi ;
- la sûreté de la procédure — rien n'empêche de sauter une étape ou de conclure trop vite ;
- la preuve — au moment du bilan, il ne reste que des souvenirs.

Le cahier des charges, écrit à partir du référentiel

OBJECTIFS

Une application métier démontrable, au cycle de vie strict et audité.

Une chaîne DevOps couvrant les trois compétences obligatoires.

Une preuve d'exécution pour chaque brique.

CONTRAINTES

Windows 11 + Docker Desktop
— d'où k3d, PowerShell,
control node conteneurisé.

Projet individuel, mené en
parallèle du travail.

Zéro cloud, zéro donnée réelle,
aucune commande arbitraire.

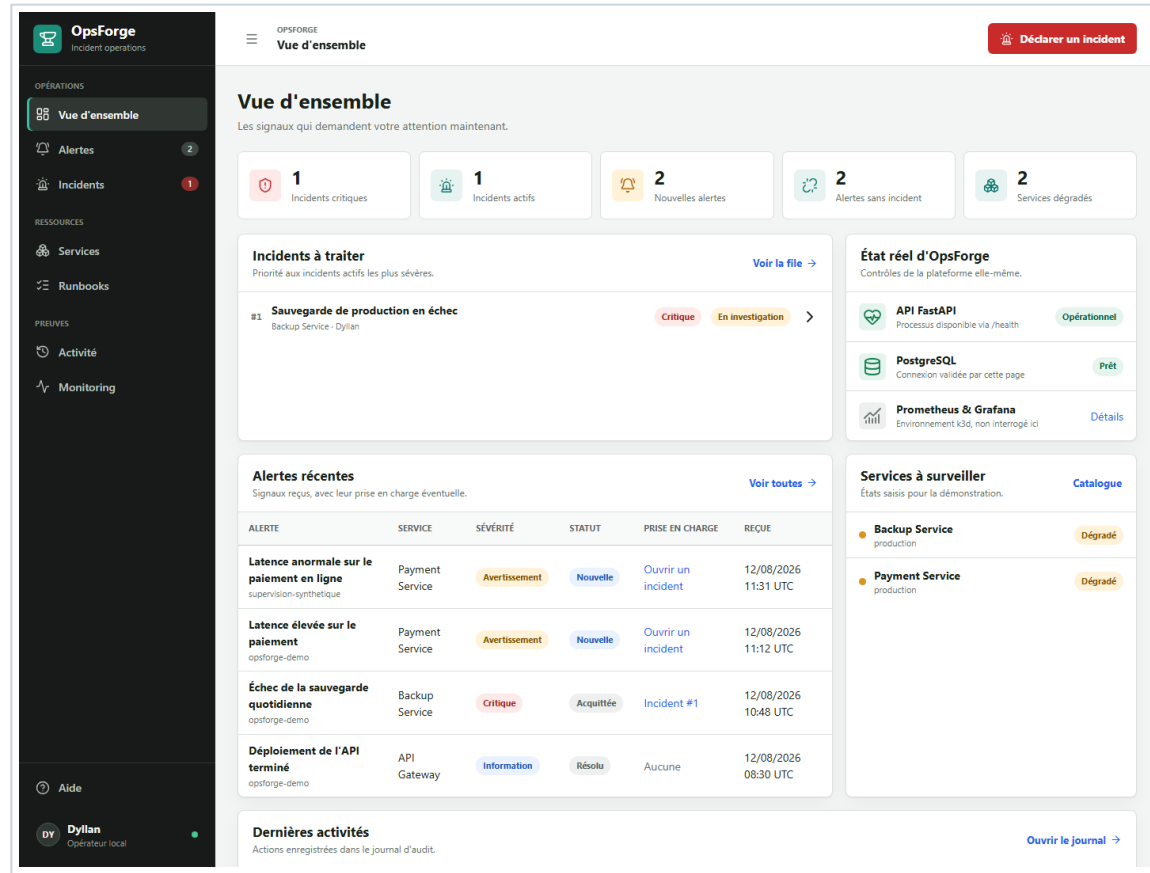
LIVRABLES

Le dépôt Git complet :
application, tests, Docker,
k8s/, ansible/, scripts, CI.

La documentation par phase et
29 décisions d'architecture.

Le dossier de projet
et ce support.

Une console d'exploitation, utilisable sans outil externe



CE QUE L'OPÉRATEUR VOIT

Les incidents à traiter,
les alertes récentes, et l'état
réel de la plateforme.

HUIT SECTIONS

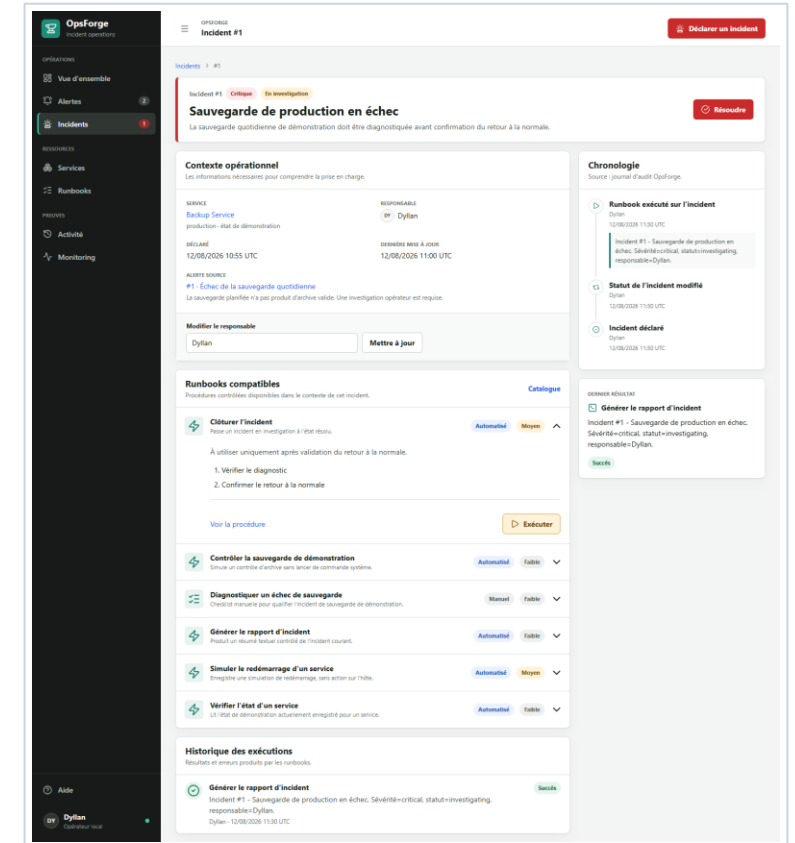
Vue d'ensemble, Alertes,
Incidents, Services, Runbooks,
Activité, Monitoring, Aide.

Tout le parcours se fait dans l'interface — pas de Swagger, pas de ligne de commande.

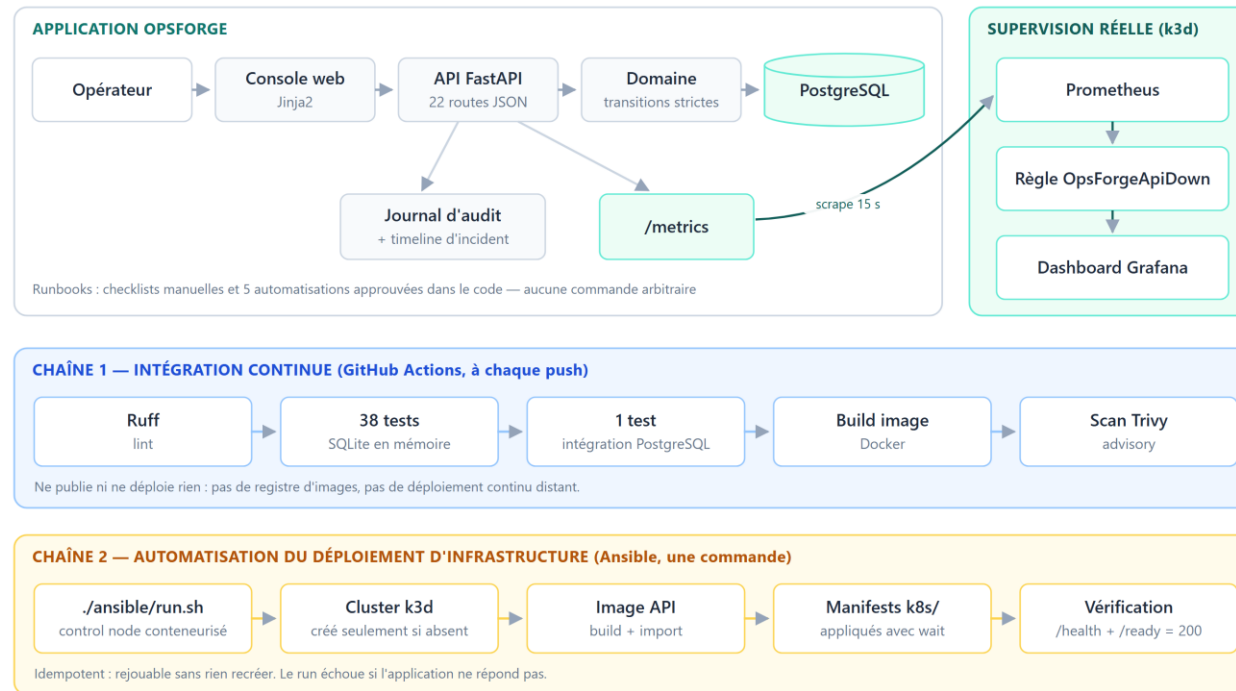
Des règles appliquées côté serveur, pas des écrans

- Transitions strictement en avant : une alerte va de « nouvelle » à « acquittée » puis « résolue », jamais en arrière. Un incident ne se rouvre pas.
- Une alerte n'a qu'un seul incident actif : le doublon est refusé, avec l'identifiant de l'incident existant.
- Aucune exécution arbitraire : cinq automatisations approuvées dans le code. Une clé inconnue est rejetée.
- Un runbook manuel ne peut pas être déclaré réussi avec une checklist incomplète.
- Toute mutation — y compris les échecs contrôlés — produit une entrée d'audit.

Chaque règle est testée négativement : le refus est vérifié, pas seulement le cas nominal.



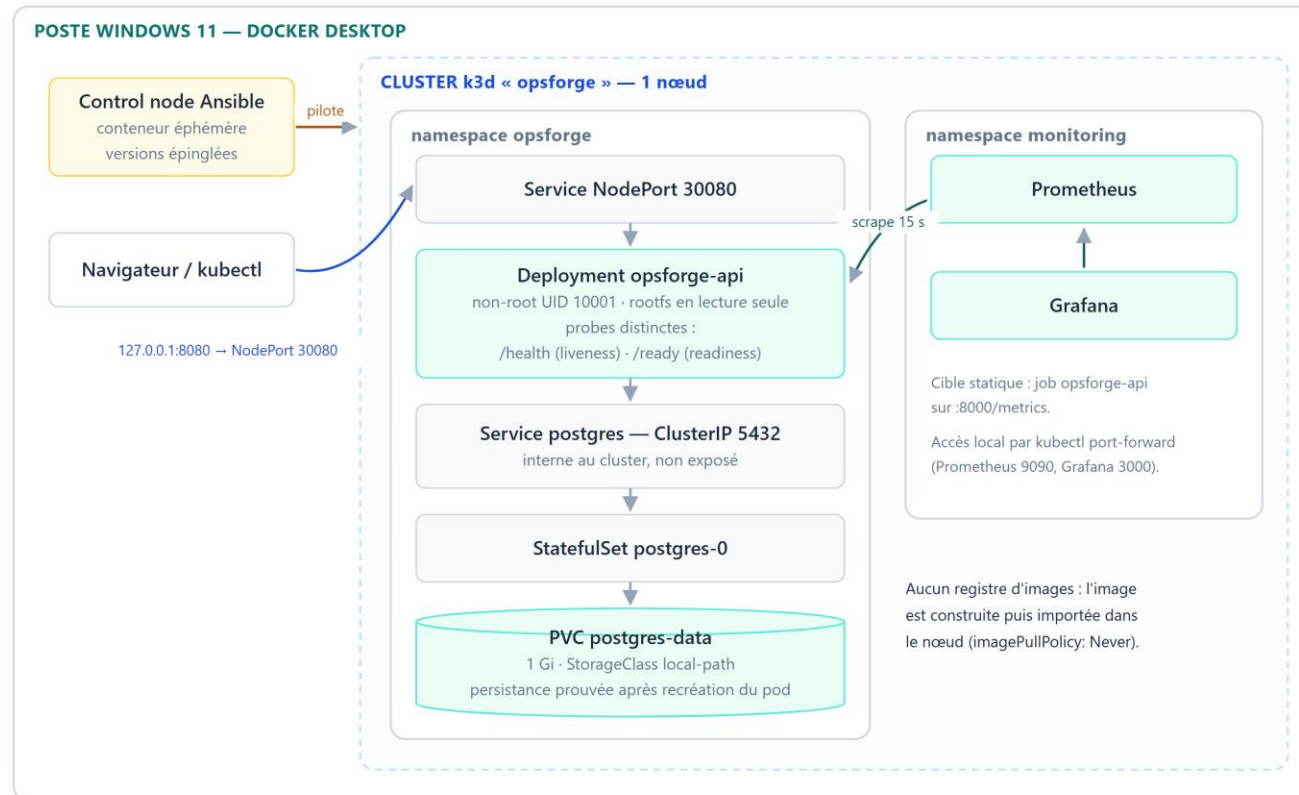
Une application, et deux chaînes d'outillage indépendantes



Les deux chaînes sont indépendantes : la CI valide le code et l'image sans rien publier ; Ansible reconstruit l'image localement et déploie l'infrastructure locale.

La CI valide le code et l'image sans rien publier. Ansible reconstruit l'image localement et déploie l'infrastructure. Aucun registre, aucun déploiement distant.

Tout tient sur un poste, et c'est reproductible



LE CLUSTER

k3d : un Kubernetes k3s dans des conteneurs Docker. Un seul nœud.

L'ACCÈS

API sur 127.0.0.1:8080 via un NodePort. Supervision par port-forward.

LA LIMITE

Mono-nœud, local. Pas de haute disponibilité, et je ne la revendique pas.

D'une séquence manuelle à une seule commande

AVANT

Une séquence manuelle documentée :
créer le cluster k3d, construire et
importer l'image, appliquer les
manifests un par un, attendre,
vérifier à la main que ça répond.

Documenté — mais pas automatisé.

APRÈS

Une seule commande, idempotente :

cluster créé seulement s'il n'existe pas ;
image construite et importée ;
manifests appliqués dans l'ordre, chaque
étage attendu prêt ;
puis /health et /ready vérifiés en HTTP.

```
$ ./ansible/run.sh deploy.yml -e cluster_name=opsforge-ansible-test ...
```

PLAY RECAP	ok=22	changed=9	failed=0	"/health -> 200"	"/ready -> 200"
second run	ok=21	changed=2	failed=0	(idempotence)	
teardown	ok=3	changed=1	failed=0		

Gérer des containers : durcis, et une persistance prouvée

LE DURCISSEMENT, VÉRIFIÉ SUR CLUSTER RÉEL

```
securityContext:
  runAsNonRoot: true
  runAsUser: 10001
  readOnlyRootFilesystem: true
  capabilities: {drop: ["ALL"]}
  seccompProfile: {type: RuntimeDefault}

readinessProbe: /ready    livenessProbe: /health
```

Les deux sondes sont différentes, et c'est le point clé : /ready retire l'API du service quand la base est injoignable ; /health ne redémarre que si le processus est mort.

LA PERSISTANCE, PROUVÉE — PAS SUPPOSÉE

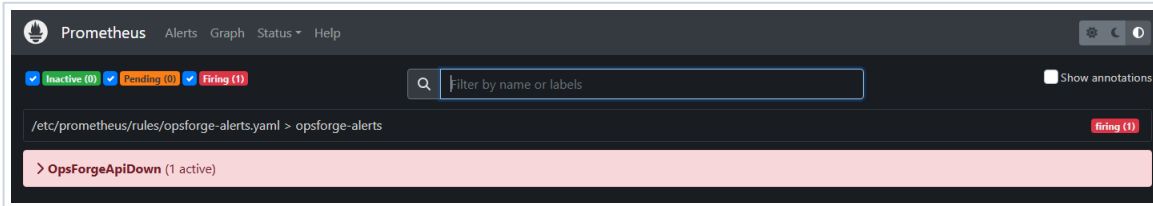
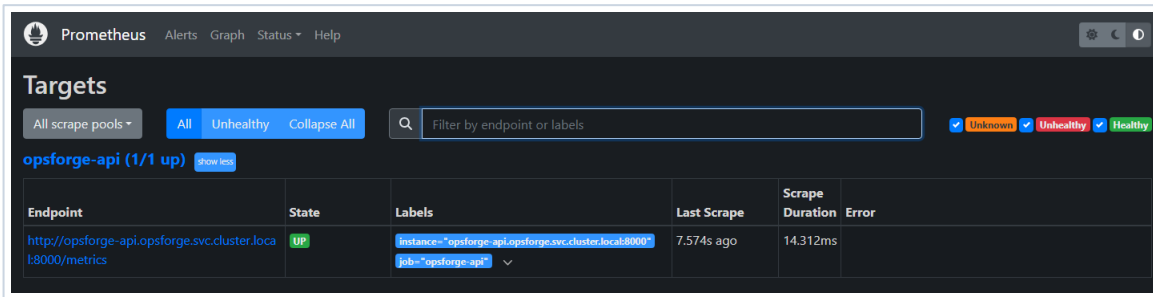
```
pod UID avant : 554c3801-0b0e-42b7-...
  INSERT INTO persistence_check VALUES ('...');

$ kubectl delete pod postgres-0
  le StatefulSet recree le pod

pod UID apres : e035ce38-526f-4b3b-...    (pod neuf)
SELECT marker : dossier-v2-persisted    (donnee la)
```

Limite : la StorageClass local-path reste locale au nœud. Ce n'est pas du stockage réseau — et je le dis avant qu'on me le demande.

Exploiter une supervision : l'alerte s'est déclenchée

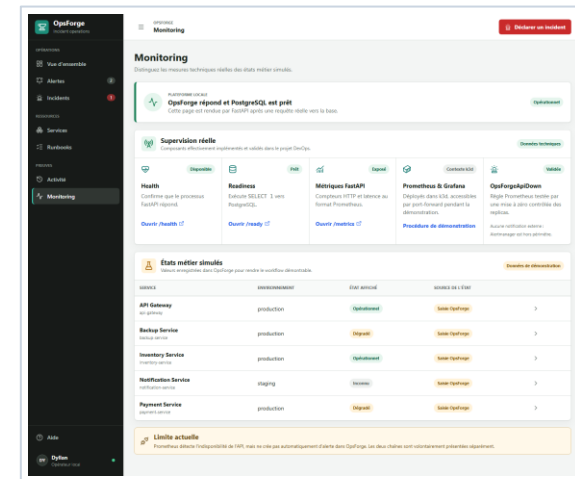


```
$ kubectl scale deploy/opsforge-api --replicas=0
t+20s up=0 inactive
t+40s up=0 pending
t+70s up=0 FIRING <<<
$ kubectl scale deploy/opsforge-api --replicas=1
t+40s up=1 inactive (resolu)
```

LE POINT D'HONNÊTÉTÉ DU PROJET

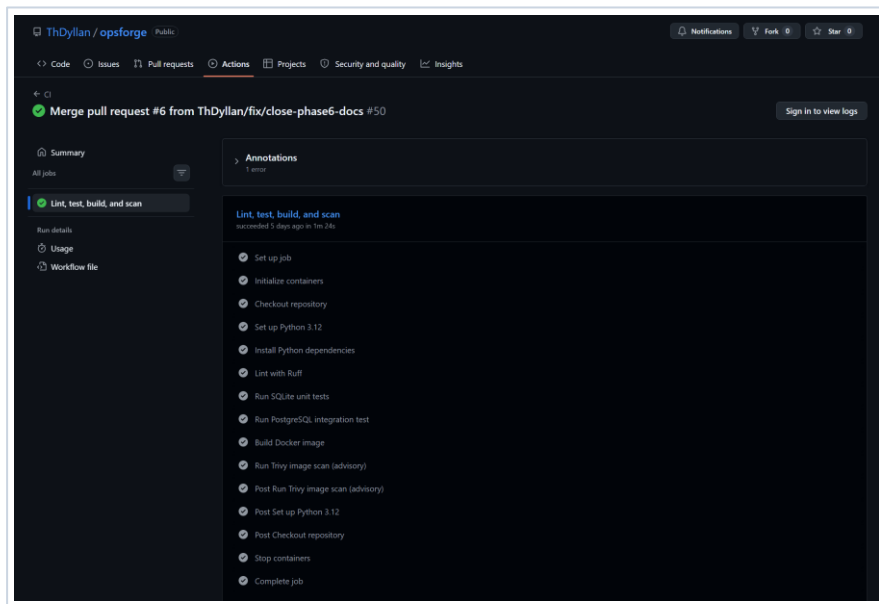
Prometheus supervise OpsForge lui-même — l'API déployée dans le cluster.

Les états métier des services du catalogue sont des données de démonstration, et la console l'affiche explicitement.



Autour : intégration continue et sauvegardes restaurables

À CHAQUE PUSH — ET UN SIGNAL QU'ON NE MASQUE PAS



Job vert — et pourtant une annotation d'erreur : le scan Trivy signale ses vulnérabilités, sans bloquer la livraison. Politique advisory, assumée.

LA CHAÎNE CI

Ruff → 38 tests SQLite →
1 test d'intégration PostgreSQL
→ build de l'image → scan.
Base de test éphémère, supprimée.

SAUVEGARDE

pg_dump scripté, archive vérifiée.
Restauration PAR DÉFAUT dans une
base temporaire — la principale exige
un drapeau et la saisie de RESTORE.

INCIDENT	SERVICE	SÉVÉRITÉ	RESPONSABLE	DATE DERNIÈRE ACTIVITÉ
S...	Backup Service	Critique	DY	12/08/2026 15:46 UTC

Depuis l'alerte #1

ID	INCIDENT	SERVICE	SÉVÉRITÉ
#1	Sauveg... de produc... en échec Depuis l'alerte #1	Backup Service	Critique
	Latence API réfection		

Corrigés en branches dédiées, couverts par trois tests de non-régression : la suite est passée de 35 à 38 tests.

Les trois compétences obligatoires, et ce que je n'affirme pas

CP n° 2 **Automatiser le déploiement d'une infrastructure**

Playbook Ansible : toute l'infrastructure locale déployée et vérifiée en une commande idempotente.

CP n° 7 **Gérer des containers**

Image durcie non-root, orchestration Compose puis Kubernetes, persistance prouvée après destruction du pod.

CP n° 10 **Exploiter une solution de supervision**

Application instrumentée, scrape réel, dashboard, et une alerte observée de « inactive » à « firing » puis résolue.

Deux nuances que j'assume plutôt que de les contourner : le stockage reste local au nœud, et « les échanges avec les développeurs » n'ont pas de sens dans un projet individuel — la boucle existe, elle est tracée dans mes décisions.
Non revendiquées : la mise en production dans le cloud, et l'épreuve d'anglais, évaluées ailleurs.

Le fichier de configuration silencieusement ignoré

LE SYMPTÔME

« skipping: no hosts matched »

Aucun hôte trouvé — alors que l'inventaire existait, déclaré dans ansible.cfg.

LE DIAGNOSTIC

Inventaire mal écrit ? Non.
Mauvais répertoire ? Non.

ansible --version affichait
« config file = None ».

LA CAUSE

Ansible refuse un ansible.cfg
situé dans un répertoire
inscriptible par tous.

Un dépôt Windows monté dans
Docker est world-writable.

LA CORRECTION

ANSIBLE_CONFIG fixé dans
l'image — la solution prévue
par l'outil.

Plus un inventaire explicite,
par ceinture et bretelles.

Le second enseignement, le plus important : en corrigeant, j'ai découvert que mes validations initiales avaient été faites avec des commandes ajustées à la main — le script documenté, lui, ne les portait pas. Quiconque l'aurait lancé aurait reproduit l'échec.

J'ai aligné le script, tout re-testé en ne passant que par lui, et épinglé les versions validées.

Les limites, nommées — et ce qu'elles appellent ensuite

DOMAINE	CE QUE JE NE REVENDIQUE PAS	L'ÉVOLUTION NATURELLE
Infrastructure	k3d mono-nœud, local. Pas de cloud.	Kubernetes managé + Terraform
Livraison	Intégration continue, mais pas de déploiement continu distant.	Registre d'images + déclenchement par la CI
Stockage	PVC local au nœud, pas de stockage réseau.	StorageClass NFS ou CSI
Application	Pas d'authentification : les acteurs sont déclaratifs.	Authentification et rôles
Supervision	Alerte visible dans Prometheus, non routée. Métriques techniques.	Alertmanager, métriques métier
Sécurité	Trivy advisory : 19 HIGH / 3 CRITICAL visibles, sans correctif.	Seuil de blocage explicite, TLS

Ce sont des choix de périmètre, écrits dans le dépôt avant cette soutenance — pas des oublis découverts en la préparant.

Ce que je retiens, et où en est le projet

MES SATISFACTIONS

La méthode : six phases finies, validées et datées une à une.

Le projet a survécu sans dégât à un changement de poste de travail.

La correction de trajectoire : constater qu'un déploiement documenté mais manuel ne prouvait rien, et fermer l'écart avant l'examen.

MES DIFFICULTÉS

Le débogage du control node Ansible — le plus formateur, avec sa leçon sur l'artefact documenté.

La revue visuelle, impossible à automatiser, qui a révélé deux défauts réels.

Et en continu : tenir le périmètre.

Le projet est gelé au commit a9ec694 : c'est cet état, reproductible et documenté, que je vous présente. Le dépôt est public — chaque preuve de ce dossier est rejouable.

ANNEXES

Slides de secours

Questions probables, chiffres, détails techniques

Mon dossier professionnel et OpsForge se répondent

COMPÉTENCE OBLIGATOIRE	DANS MON EXPÉRIENCE PROFESSIONNELLE	DANS OPSFORGE
CP n° 2 Automatiser le déploiement d'une infrastructure	Automatisation PowerShell de provisionnement (comptes, dossiers, droits) — pas du déploiement d'infrastructure	Playbook Ansible : l'infrastructure complète en une commande idempotente et auto-vérifiée
CP n° 7 Gérer des containers	Aucun exemple : rien dans mon parcours professionnel ne porte sur des containers	Image durcie, Compose puis Kubernetes, persistance du stockage prouvée
CP n° 10 Exploiter une solution de supervision	Supervision multi-clients réelle, en équipe — et un incident où le vert de la console cachait une panne bien réelle	Chaîne Prometheus/Grafana et une alerte réellement déclenchée puis résolue

Les deux documents ne se répètent pas : ils se complètent. Le projet apporte ce que l'expérience ne pouvait pas démontrer.

Questions probables — le périmètre

« Pourquoi pas de cloud ? »

Budget zéro cloud assumé, et le référentiel n'exige pas le cloud pour les trois compétences obligatoires. La mise en production cloud n'est pas revendiquée.

« Votre stockage n'est pas distant. »

Exact. Le conteneur est connecté à un stockage externalisé de son cycle de vie, porté par l'hôte. En k3d, local-path reste local au nœud. Une StorageClass NFS ou CSI la remplacerait sans toucher au manifest.

« Où est le déploiement continu ? »

Il n'y en a pas. La CI valide le code et l'image, elle ne publie rien. Le déploiement est automatisé mais déclenché à la main. La suite serait un registre puis un déclenchement par la CI.

« Une seule réplique, pas de HA ? »

Non. Une réplique de chaque workload. La haute disponibilité n'était pas l'objectif : je préférais prouver la persistance et le durcissement.

Questions probables — la technique

« Vos tests couvrent quoi exactement ? »

38 tests SQLite en mémoire pour le domaine, plus un test d'intégration contre un vrai PostgreSQL en CI. Ils testent surtout les refus : transitions invalides, doublons, clés d'automatisation inconnues, checklist incomplète.

« Pourquoi SQLite si vous tournez sur PostgreSQL ? »

Pour la vitesse de retour. Le risque — un comportement propre à PostgreSQL — est couvert par le test d'intégration qui rejoue le flux complet sur postgres:16-alpine dans la CI.

« Comment gérez-vous les secrets ? »

Aucun secret versionné. Le fichier .env et le Secret Kubernetes local sont ignorés par Git ; Ansible génère le Secret au déploiement depuis des variables, surchargeables par ansible-vault.

« Votre image est-elle sûre ? »

Elle est reproductible — base épinglée par digest, non-root, rootfs en lecture seule. Elle n'est pas exempte de vulnérabilités : 19 HIGH et 3 CRITICAL Debian sans correctif disponible, visibles dans la CI.

Les chiffres et les versions

LE PROJET

45 commits sur la branche du candidat

6 commits de merge, 6 pull requests

29 décisions d'architecture

6 phases validées et datées

Candidat gelé : a9ec694

LES TESTS

38 tests unitaires SQLite

1 test d'intégration PostgreSQL

Base éphémère au nom unique

Lint Ruff en amont

CI verte sur le candidat final

LA PLATEFORME

Python 3.12, FastAPI, SQLAlchemy 2

PostgreSQL 16 partout

k3d 5.9 / k3s 1.35, kubectl 1.34

Prometheus 2.55, Grafana 11.3

ansible-core 2.17, kubernetes.core 6.5

Le domaine en détail

ALERTE

```
new -> acknowledged -> resolved
new -> resolved      (direct)
transition arriere   -> 409
```

INCIDENT

```
open -> investigating -> resolved
pas de reouverture      -> 409
```

REFUS TESTES

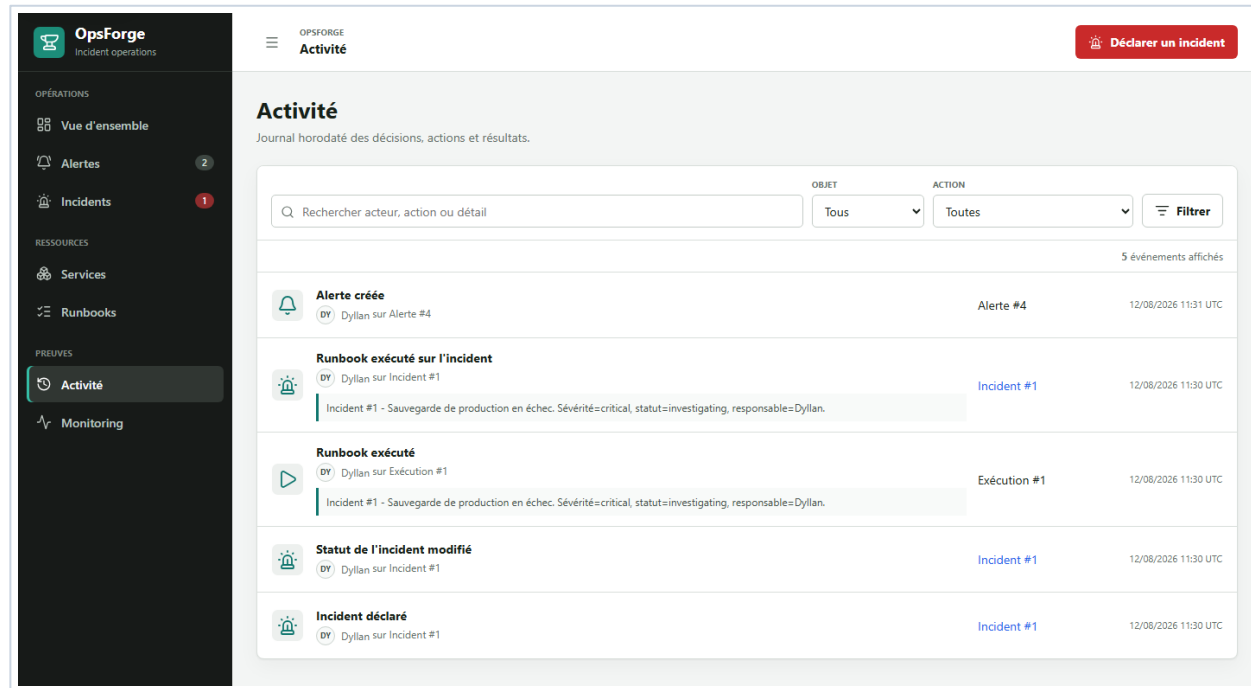
```
2e incident actif sur l'alerte -> 409
runbook manage modifie par API -> 409
cle d'automatisation inconnue  -> 422
checklist incomplete en succes -> refus
incident resolu modifie        -> 409
```

... et l'echec controle est audite

Six objets : Service · Alert · Incident · Runbook · RunbookExecution · AuditLog

L'incident hérite du service de son alerte source. Résoudre l'incident ne résout pas l'alerte : le cycle du signal et celui de la prise en charge sont indépendants — c'est un choix de modélisation, pas un oubli.

La traçabilité, vue de l'application



The screenshot displays the 'Activité' (Activity) page in the OpsForge application. The left sidebar shows navigation options: 'Vue d'ensemble', 'Alertes', 'Incidents', 'Services', 'Runbooks', 'Monitoring', and 'Activité' (selected). The main content area shows a list of events with the following columns: 'OBJET' (Object), 'ACTION' (Action), and 'DATE' (Date). The events listed are:

OBJET	ACTION	DATE
Alerte créée	Alerte #4	12/08/2026 11:31 UTC
Runbook exécuté sur l'incident	Incident #1	12/08/2026 11:30 UTC
Runbook exécuté	Exécution #1	12/08/2026 11:30 UTC
Statut de l'incident modifié	Incident #1	12/08/2026 11:30 UTC
Incident déclaré	Incident #1	12/08/2026 11:30 UTC

Chaque entrée porte l'acteur, l'action, l'objet lié et l'horodatage. Les exécutions de runbook sont auditées qu'elles réussissent ou non.